

Actividad | 3# | Auditoría Informática

Cross Site Scripting (XSS)

Ingeniería en Desarrollo de Software



TUTOR: Jessica Hernández Romero

ALUMNO: Casandra Montserrat Ortiz Cortes G-1

FECHA: 16/05/2026

Índice

● Portada	1
● Índice	2
● Introducción	3
● Descripción.....	4
● Justificación	5
● Etapa 1:	
○ Descripción del sitio web	
○ Ataque al sitio	
● Etapa 2:	
○ Ataque al sitio	
● Etapa 3:	
○ Ataque al sitio	
○ Conclusión	6
○ Referencias.....	7

Introducción

Como adelantábamos más arriba, se conoce como XSS a un tipo de ataque en el cual actores maliciosos logran inyectar un script malicioso en un sitio web para luego ser procesado y ejecutado. Comúnmente, este proceso que se basa en la confianza que tiene el sitio sobre la entrada de los datos, consiste en enviar la URL con el payload precargado al usuario víctima con un objetivo determinado: robar datos personales del usuario, cookies de sesión, implementar técnicas de ingeniería social, entre otras.

Suele ser inyectado en un parámetro de la solicitud HTTP, para luego ser procesado por la aplicación web y finalmente desplegado en un punto determinado, sin algún tipo de validación o codificación de los caracteres. Se trata de la variedad de XSS más simple y el script malicioso que busca afectar el navegador de la víctima es fácilmente modificable, probablemente sin que el usuario note el ataque.

Descripción

Cross Site Scripting (XSS) es una vulnerabilidad de seguridad que ocurre cuando una aplicación web permite que se inserte código malicioso, generalmente en JavaScript, dentro de una página que será vista por otros usuarios. Este ataque puede suceder cuando el sistema no valida o filtra correctamente la información que recibe de formularios, comentarios o campos de entrada.

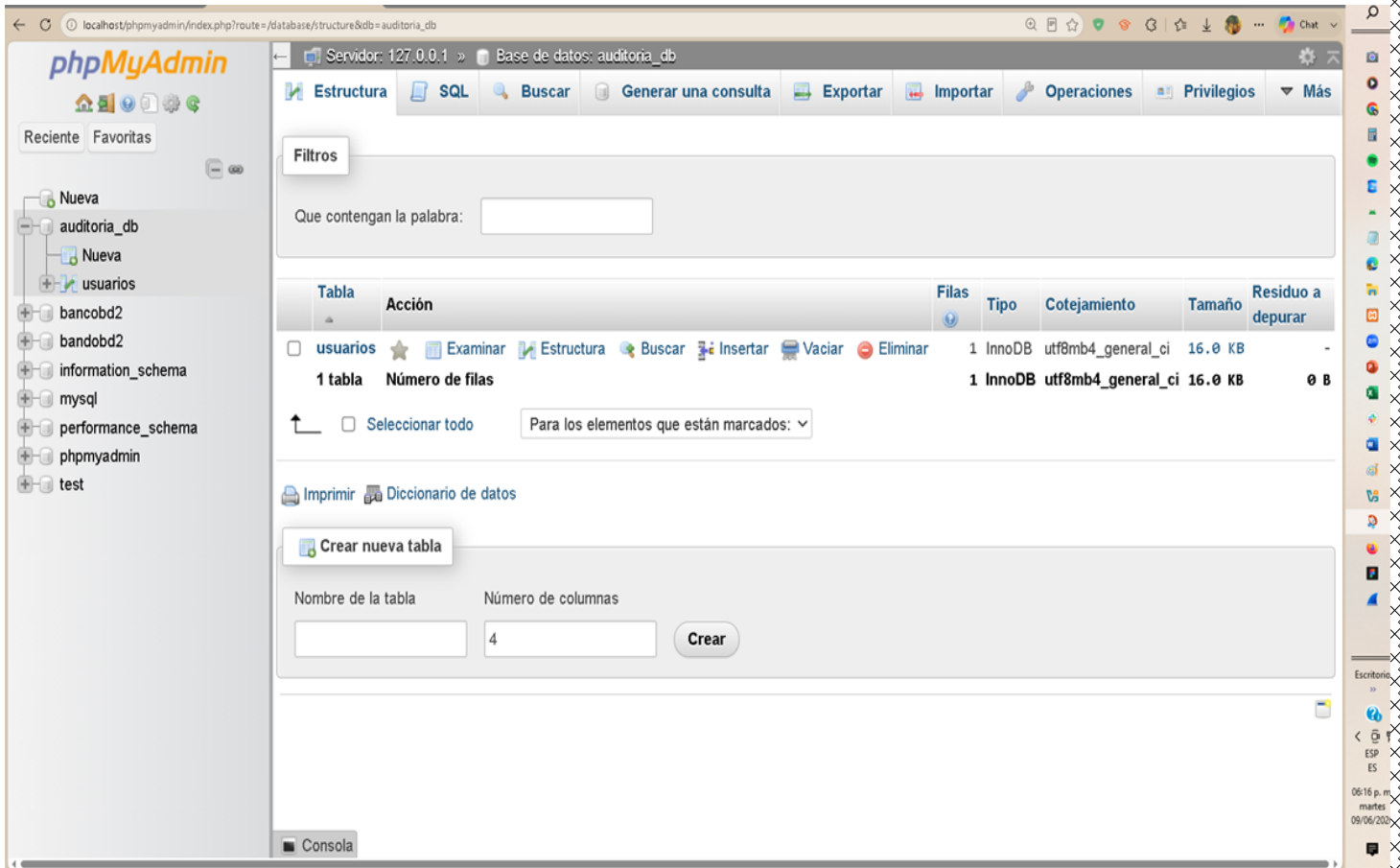
El objetivo principal de un ataque XSS es ejecutar código en el navegador de la víctima para robar información, como contraseñas, cookies de sesión o datos personales. También puede modificar el contenido de una página web o redirigir a sitios maliciosos sin que el usuario lo note.

Para prevenir este tipo de vulnerabilidad es importante validar y sanitizar los datos ingresados por los usuarios, utilizar mecanismos de codificación de salida y aplicar buenas prácticas de desarrollo seguro. De esta forma se protege la información y se mejora la seguridad de las aplicaciones web.

Justificación

Si te has preguntado si un atacante únicamente podría realizar un defacement del sitio en el que estas navegando, robar algunos datos que estén utilizando en la aplicación web vulnerable, o robar cookies de sesión —tal como sucedió en el ataque reciente que derivó en el robo del código fuente de FIFA 21 en el que utilizaron como vector inicial cookies de sesión de un canal de Slack—, la realidad indica que no. Existen diversos ataques por medio del uso de JavaScript. Por ejemplo, es posible leer la memoria de algunos procesos del usuario, como en el ataque Rowhammer que tiene como víctima a la tecnología DDR4, realizar ataques de ransomware, exfiltrar datos sensibles en un documento PDF, o realizar escaneo de la red. Dependiendo el vector vulnerable, una vulnerabilidad de XSS podrían derivar en cualquiera de los ataques antes mencionados, medidas para mitigar cualquier tipo de Cross-Site Scripting.

- Etapa 1:
 - Descripción del sitio web



Se muestra la creación de la base de datos llamada auditoria_db en phpMyAdmin. Esta base de datos se utilizó para almacenar la información de los usuarios registrados en el sistema web desarrollado para la práctica

localhost/phpmyadmin/index.php?route=/table/structure&db=auditoria_db&table=usuarios

phpMyAdmin

Reciente Favoritas

Nueva auditoria_db Nueva usuarios bancobd2 bandobd2 information_schema mysql performance_schema phpmyadmin test

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones

Estructura de tabla Vista de relaciones

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
1	id	int(11)			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
2	nombre	varchar(100)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
3	correo	varchar(100)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
4	password	varchar(100)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más

Seleccionar todo Para los elementos que están marcados: Examinar Cambiar Eliminar Primaria Único

Índice Espacial Texto completo Agregar a columnas centrales Eliminar de las columnas centrales

Imprimir Planteamiento de la estructura de la tabla Hacer seguimiento a la tabla Mover columnas Normalizar

Agregar 1 columna(s) después de password Continuar

Índices

Acción	Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
Editar Renombrar Eliminar	PRIMARY	BTREE	Sí	No	id	1	A	No	

Crear un índice en 1 columna(s) Continuar

Particiones

Consola

Esta la tabla usuarios creada dentro de la base de datos auditoria_db. La tabla contiene los campos id, nombre, correo y password, los cuales se utilizan para almacenar la información de los usuarios registrados en el sistema.

localhost/auditoria/registro.php

Registro de Usuario

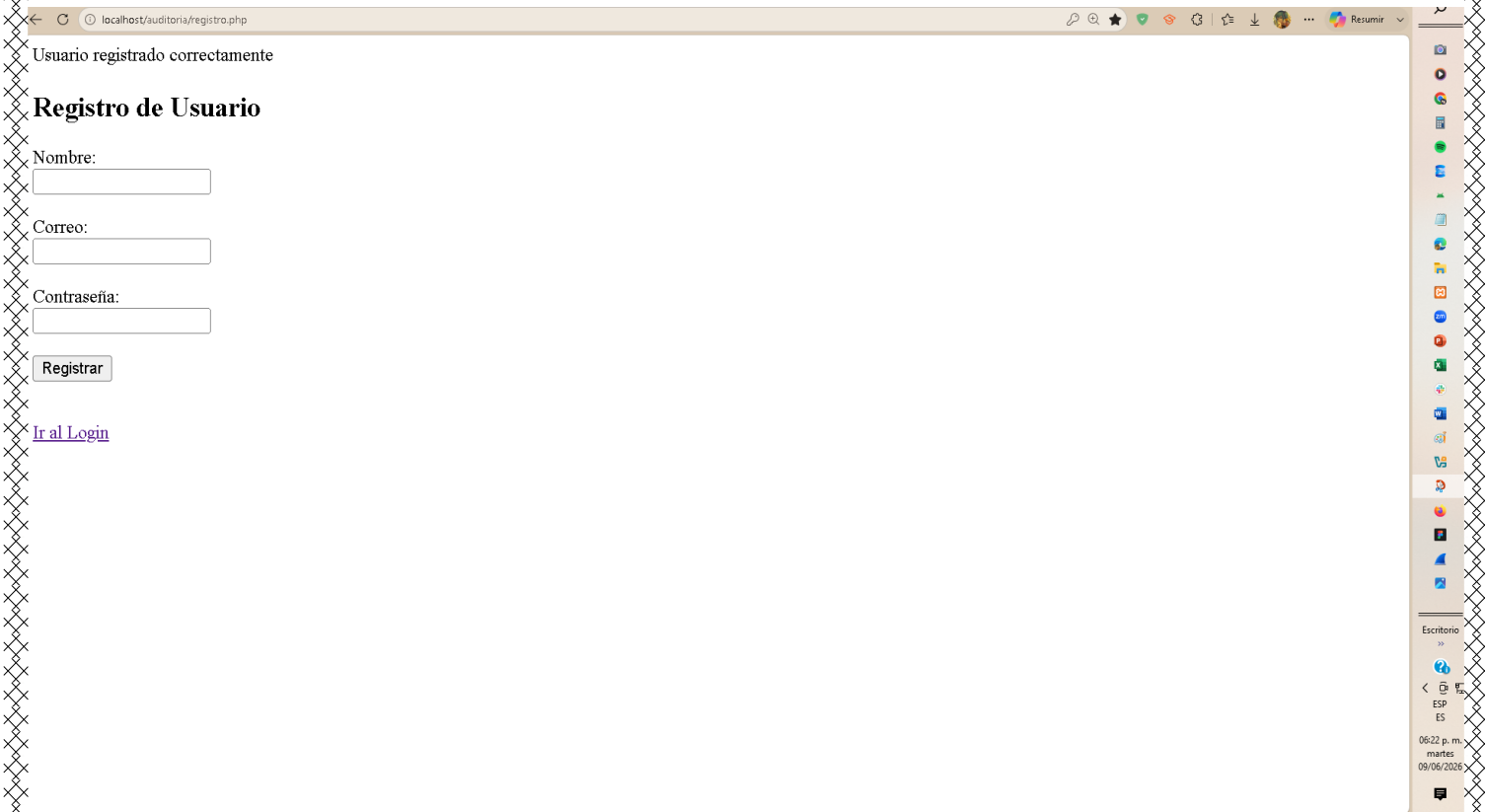
Nombre:

Correo:

Contraseña:

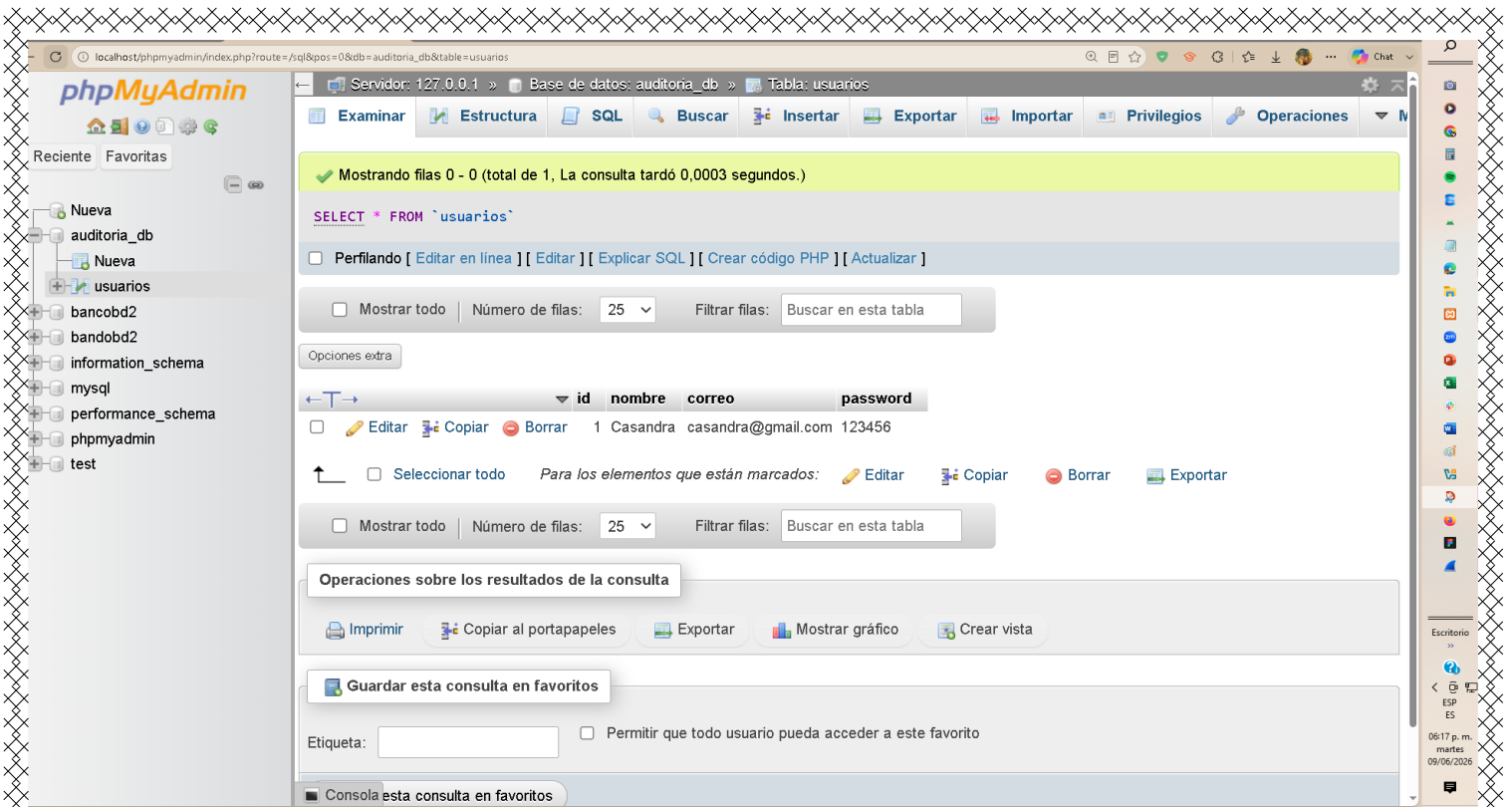
[Ir al Login](#)

Se muestra el formulario de registro de usuarios desarrollado en PHP. Este formulario permite ingresar nombre, correo electrónico y contraseña para crear una nueva cuenta dentro del sistema.

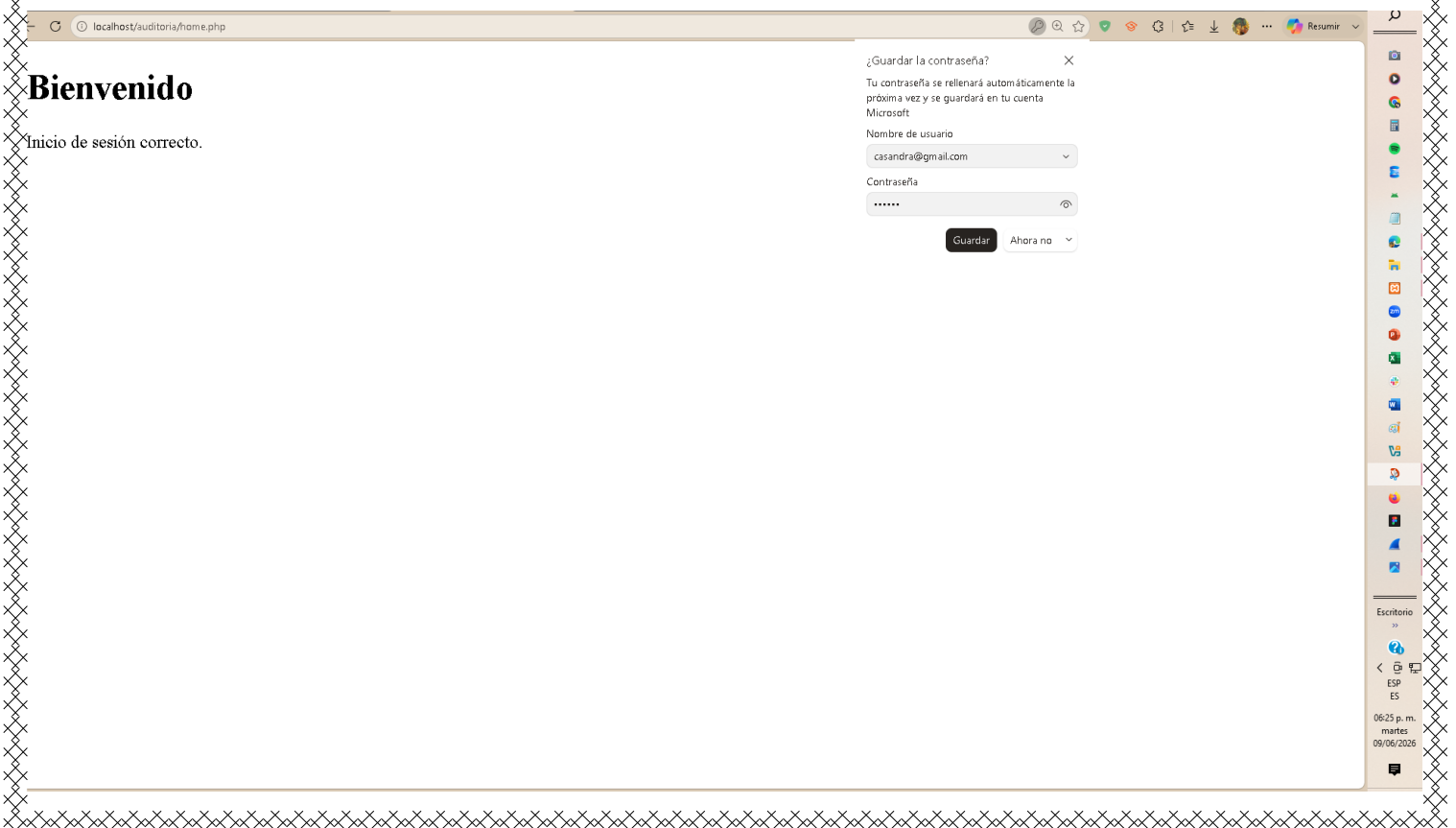


The screenshot shows a web browser window with the address bar displaying 'localhost/auditoria/registro.php'. The page content includes a success message 'Usuario registrado correctamente' at the top. Below it is the title 'Registro de Usuario' in bold. The form consists of three input fields labeled 'Nombre:', 'Correo:', and 'Contraseña:', each followed by a text box. A 'Registrar' button is positioned below the password field. At the bottom of the form, there is a link that says 'Ir al Login'. The browser's right sidebar shows a list of application icons, and the bottom status bar displays the system clock as '06:22 p. m. martes 09/06/2026'.

Observamos el mensaje de confirmación que indica que el usuario fue registrado correctamente en la base de datos. Esto demuestra que la conexión entre el formulario y MySQL funciona de manera adecuada.



Nos muestra la información del usuario almacenada en la tabla usuarios. Esto confirma que los datos ingresados durante el registro fueron guardados exitosamente en la base de datos.



Vemos la pantalla de bienvenida después de ingresar correctamente las credenciales del usuario. Esto demuestra que el sistema valida la información almacenada en la base de datos y permite el acceso cuando los datos son correctos.

o Ataque al sitio

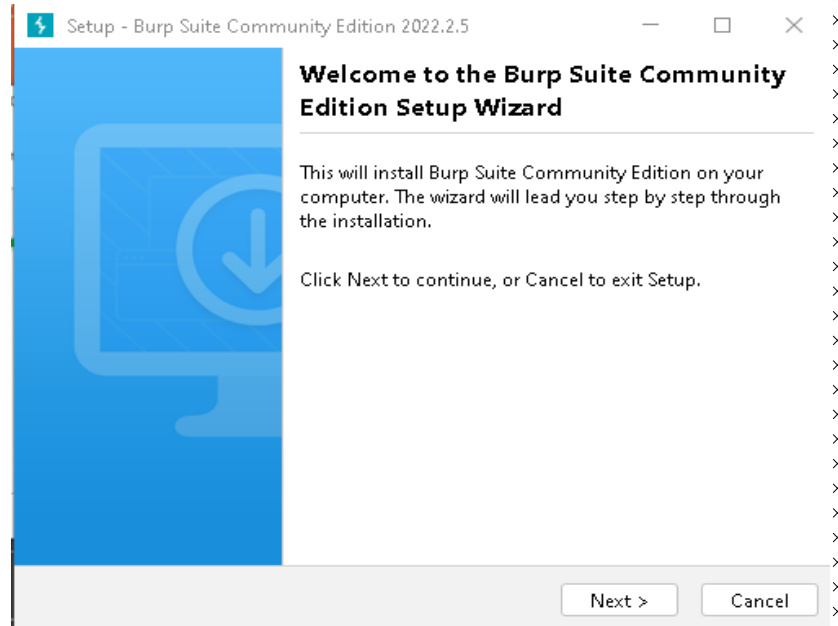
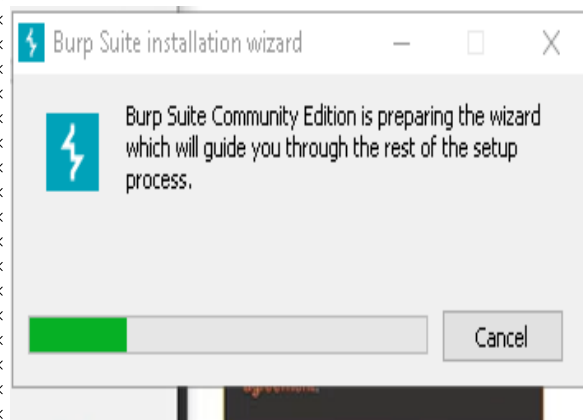
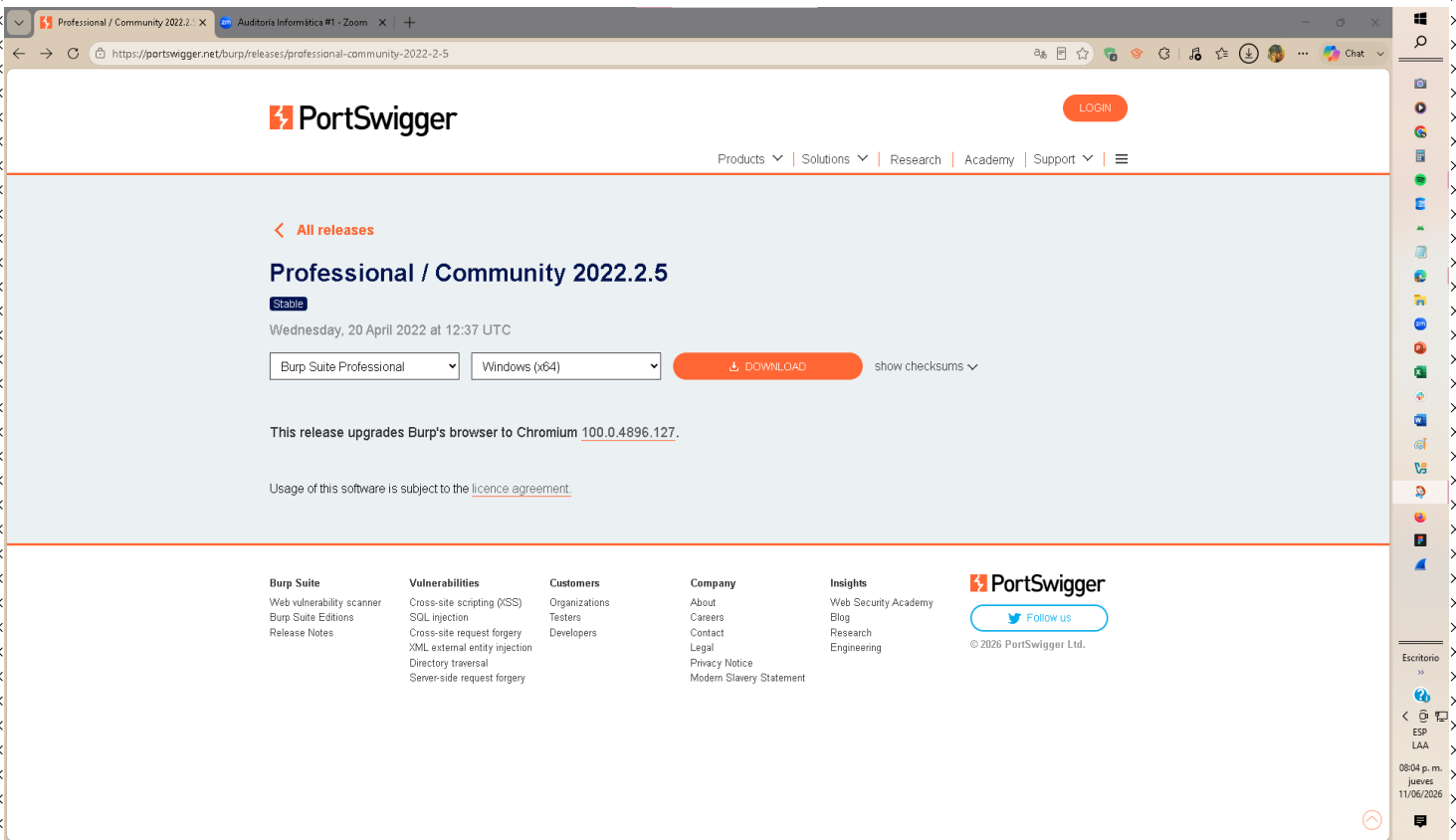
The image shows a Wireshark packet capture of a network session. The top pane displays a list of captured packets, with packet 25 selected. The middle pane shows the details of the selected packet, which is a TCP segment from 192.168.100.48 to 152.236.2.55, sequence number 55777, acknowledgment number 443, and window size 1024. The bottom pane shows the raw packet data in hexadecimal and ASCII. The packet is a TCP segment with a sequence number of 55777 and an acknowledgment number of 443. The data field contains the ASCII string "dQ: H' ... Z ... E".

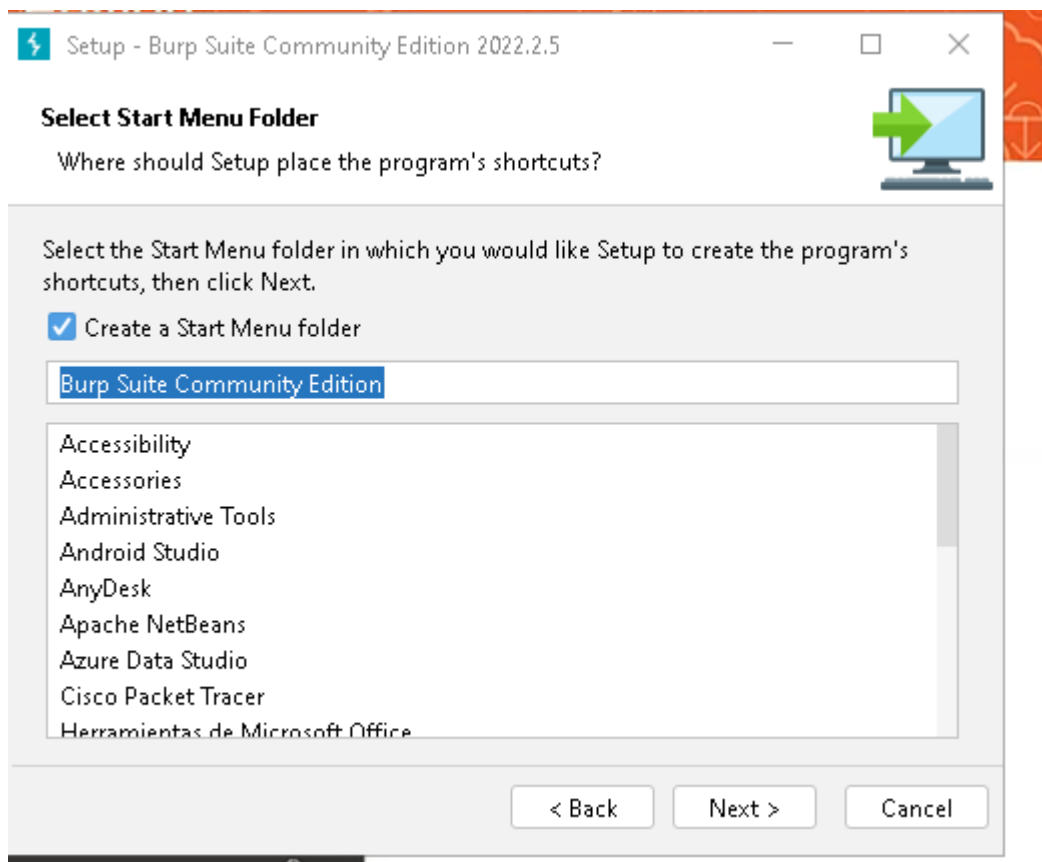
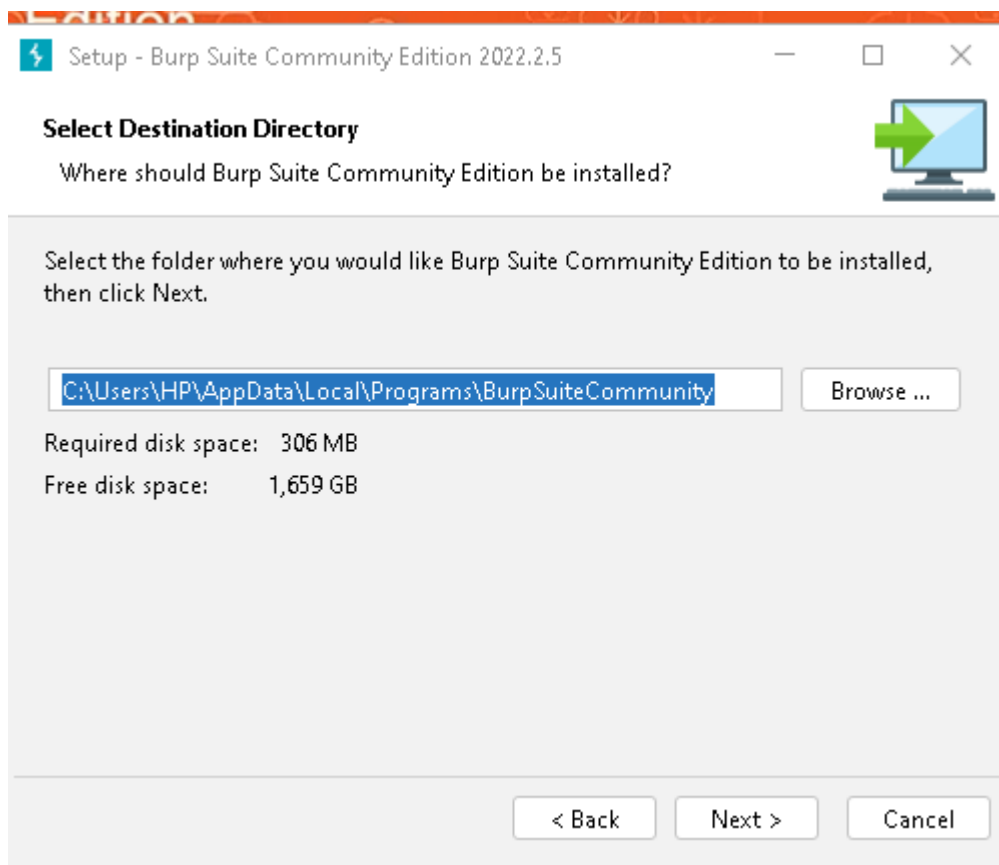
No.	Time	Source	Destination	Protocol	Length	Info
24	3.564370300	152.236.2.55	192.168.100.48	TCP	60	443 → 55777 [ACK] Seq=1 Ack=1 Win=501 Len=0
25	3.572408800	192.168.100.48	152.236.2.55	TCP	55	[TCP Keep-Alive] 55777 → 443 [ACK] Seq=0 Ack=2 Win=1024 Len=0
26	3.572408800	192.168.100.48	152.236.2.55	TCP	55	[TCP Keep-Alive] 55777 → 443 [ACK] Seq=0 Ack=2 Win=1024 Len=0
27	3.607233600	152.236.2.55	192.168.100.48	TCP	66	[TCP Previous segment not captured] 443 → 55777 [ACK] Seq=2 Ack=1 Win=501 Len=0 SLE=0 SRE=1

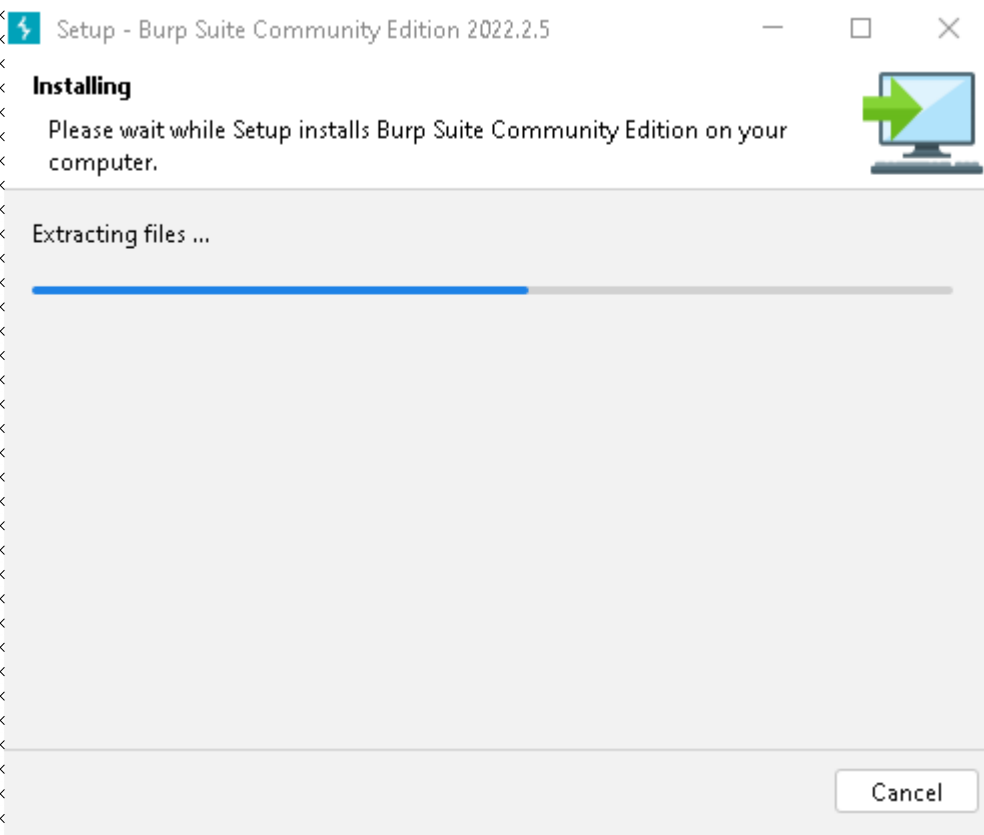
Para analizar la vulnerabilidad de pérdida de autenticación y gestión de sesiones, se utilizó Wireshark para capturar y monitorear el tráfico de red generado durante el proceso de autenticación. Se aplicó un filtro TCP para visualizar los paquetes capturados y observar la comunicación entre el cliente y el servidor.

• Etapa 2:

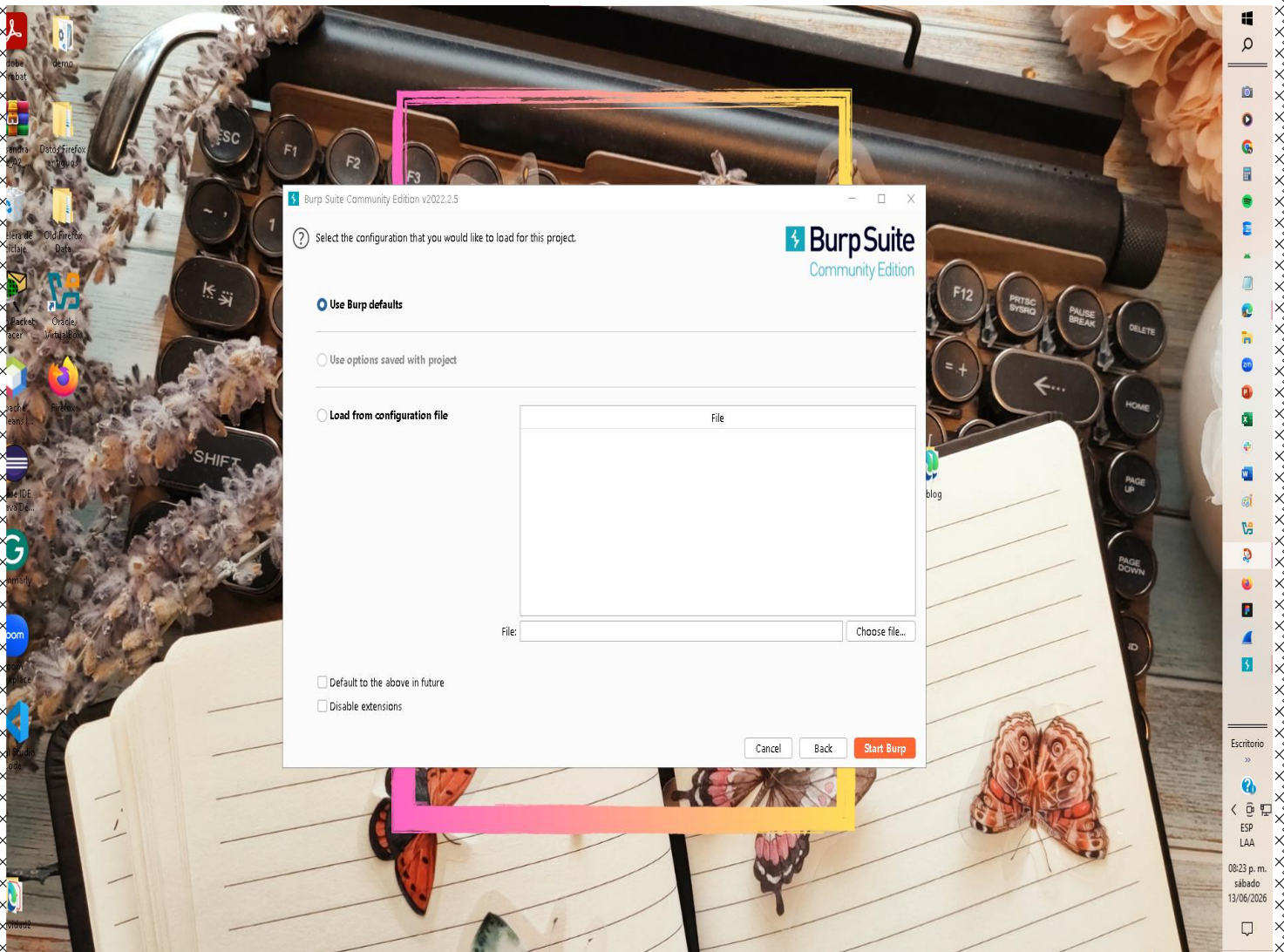
○ Ataque al sitio



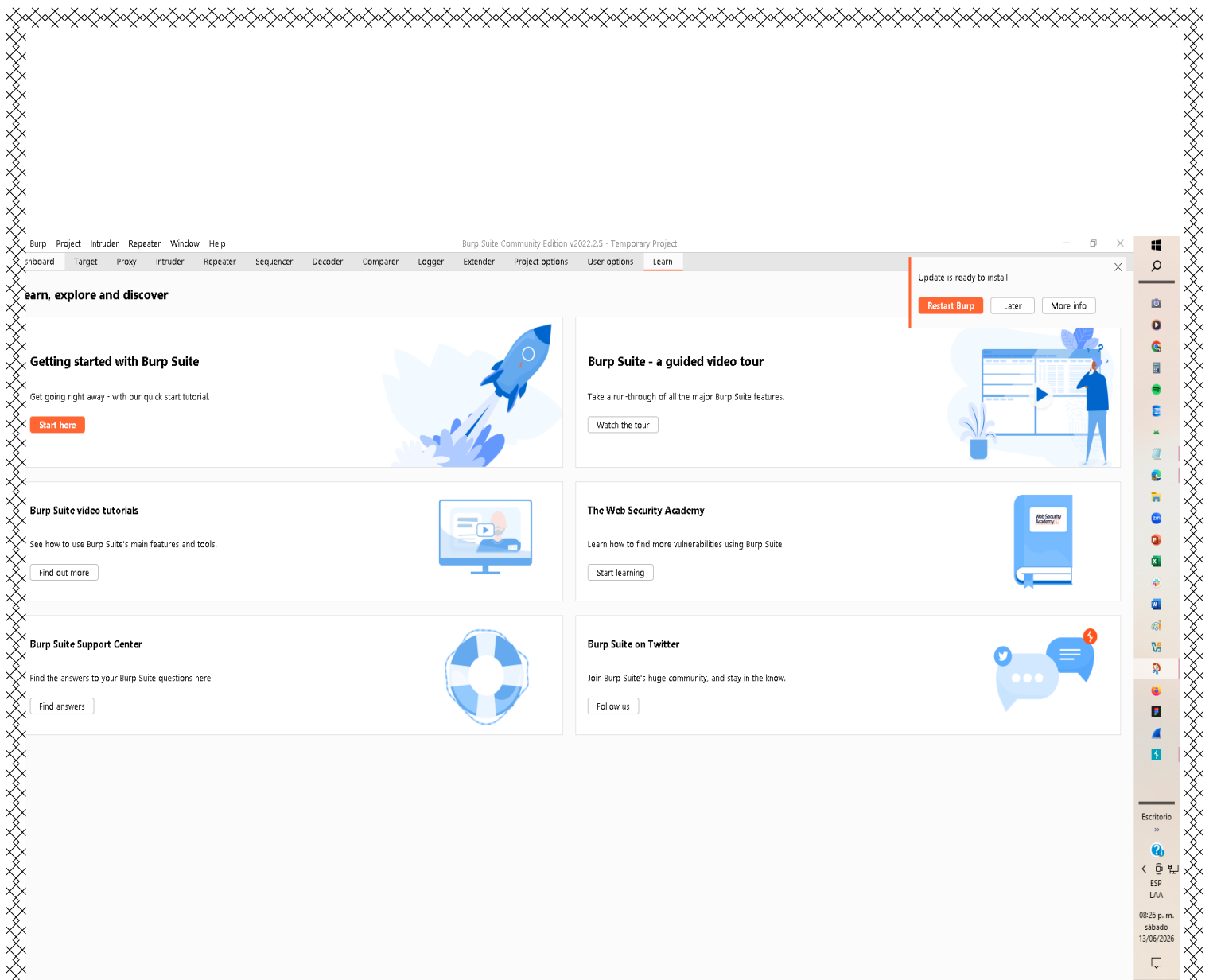




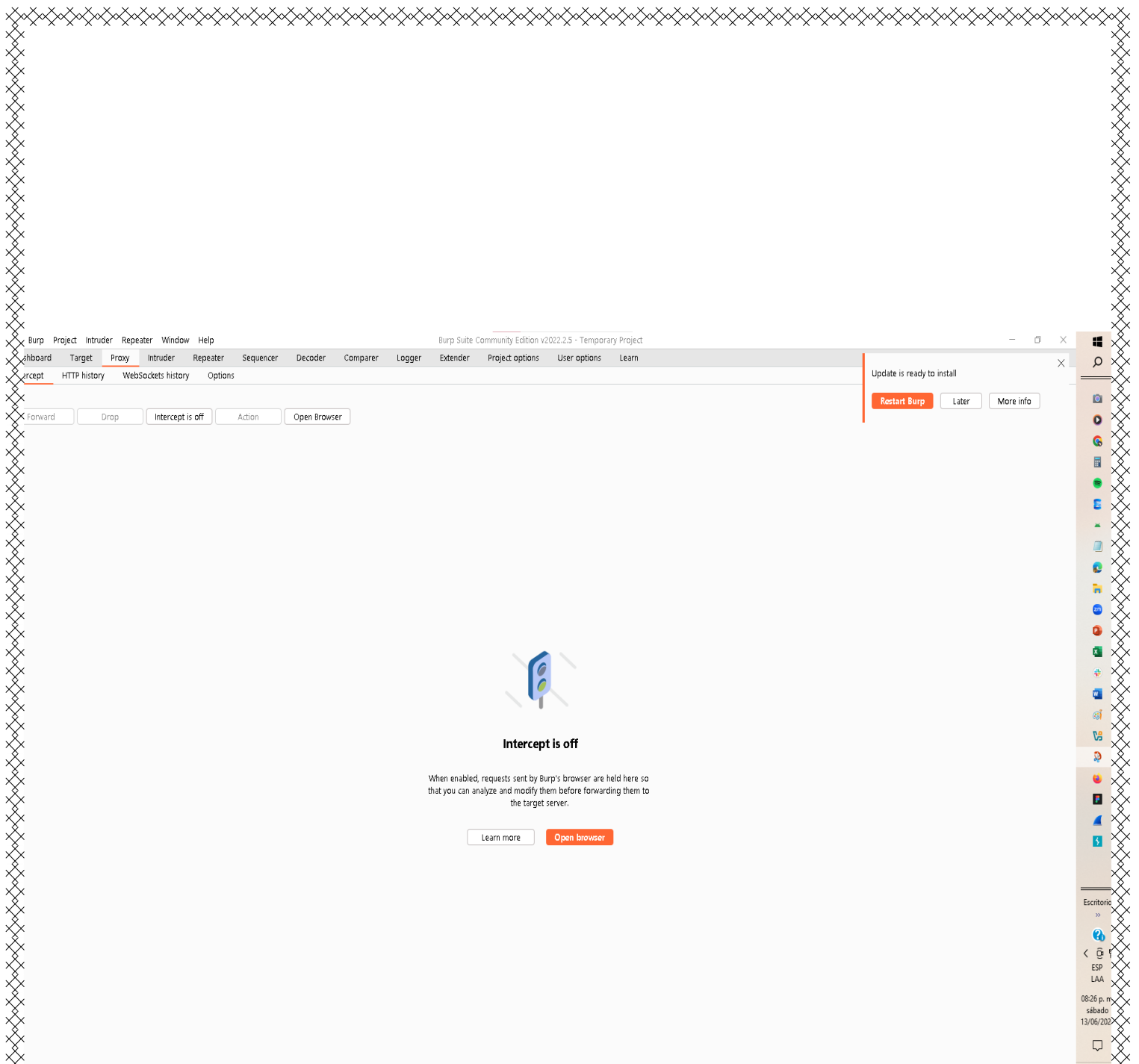
Se descargó Burp Suite Community Edition desde la página oficial para realizar la actividad.



Se creó un proyecto temporal en Burp Suite para trabajar durante la actividad



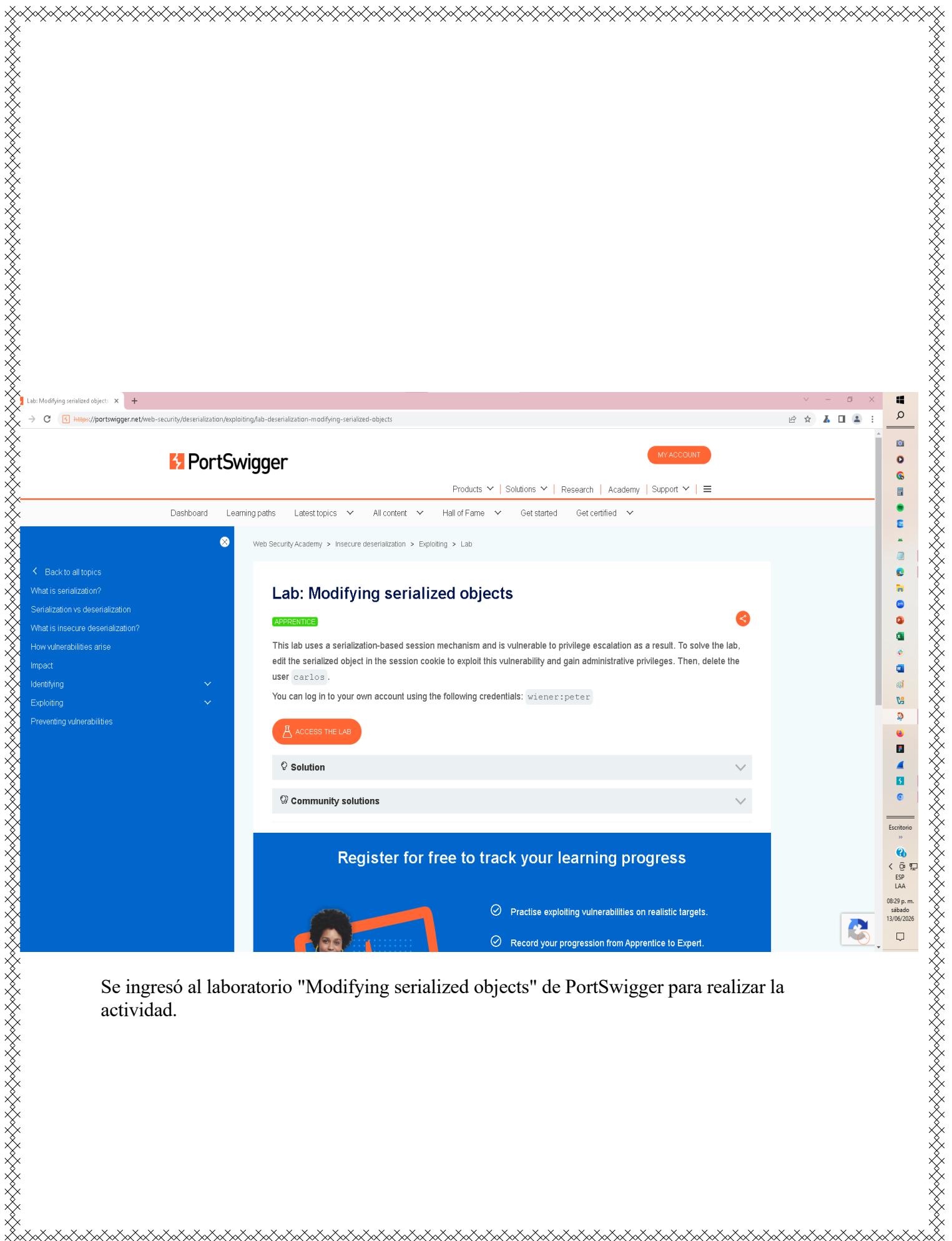
Se mostró la pantalla principal de Burp Suite con las herramientas disponibles para analizar tráfico web.



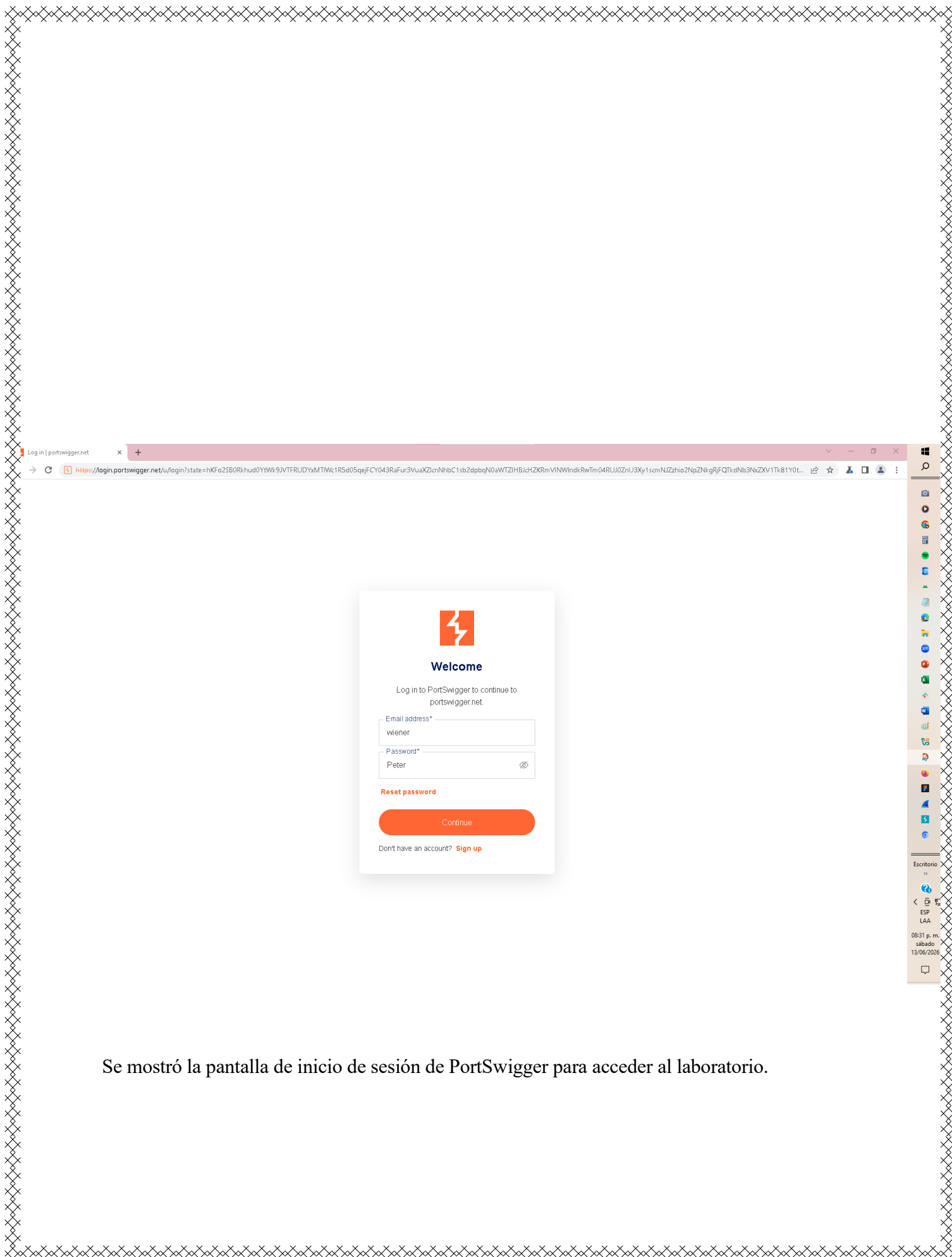
Se revisó la sección Proxy para observar y capturar las peticiones enviadas por el navegador.



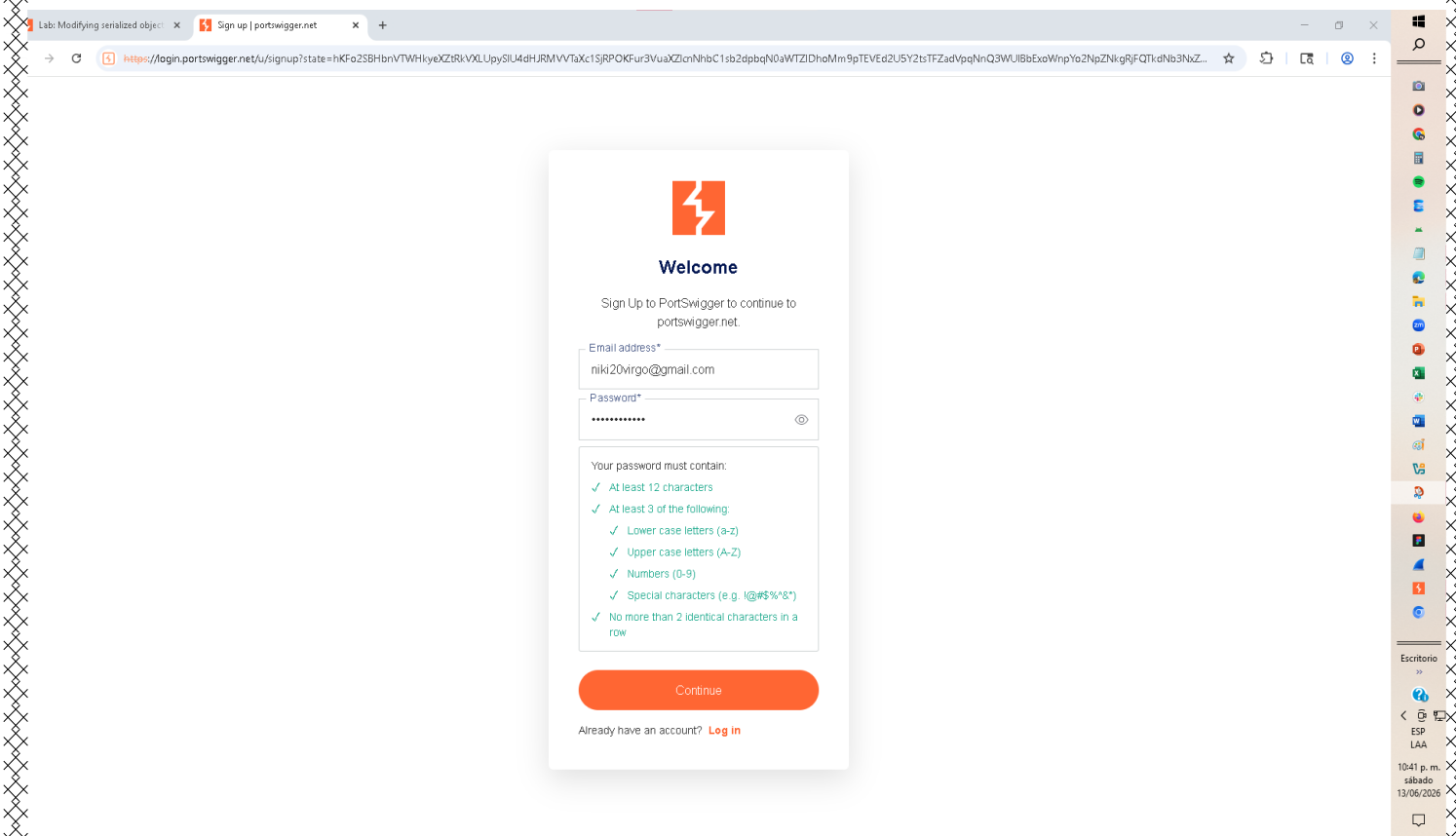
Se abrió el navegador integrado de Burp Suite para acceder al laboratorio de práctica.



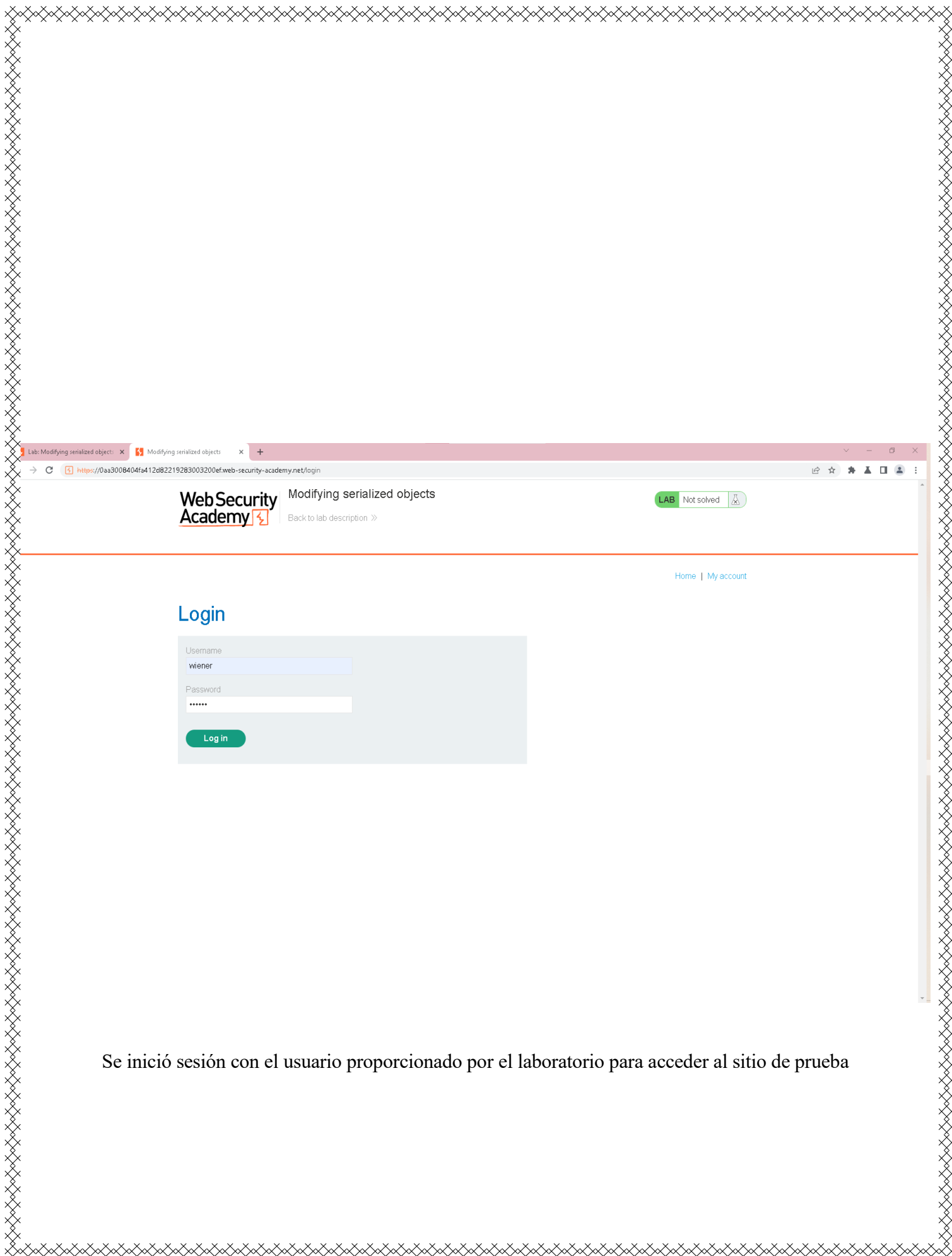
Se ingresó al laboratorio "Modifying serialized objects" de PortSwigger para realizar la actividad.



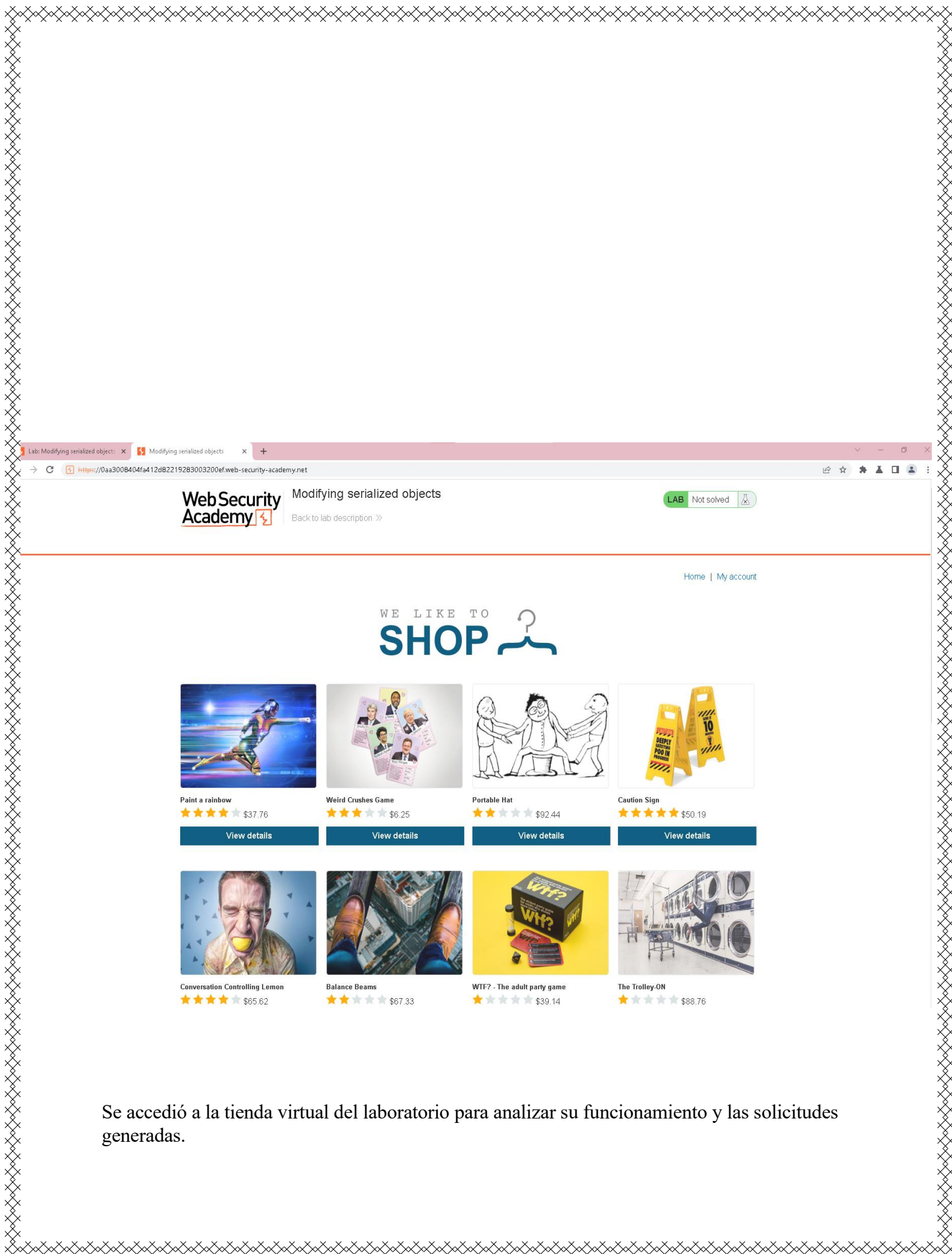
Se mostró la pantalla de inicio de sesión de PortSwigger para acceder al laboratorio.



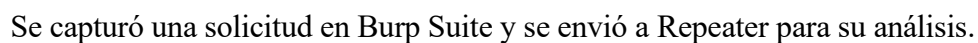
Se completó el formulario de registro para crear una cuenta y continuar con la práctica



Se inició sesión con el usuario proporcionado por el laboratorio para acceder al sitio de prueba



Se accedió a la tienda virtual del laboratorio para analizar su funcionamiento y las solicitudes generadas.



Target: https://0aa3008404fa412d82219283003200ef.web-security-academy.net HTTP/2

Request (quest):

```
GET /my-account?id=wiener HTTP/2
Host: 0aa3008404fa412d82219283003200ef.web-security-academy.net
Cookie: session=1zo00iJvC2VyiJoyOntz0gg6InVs2KJuTW1ljts0jY6Indp2W5ic17cs0i0iJhZG1phiI7Yjow030V3d
Cache-Control: max-age=0
Sec-Ch-Ua: "Not(A:Brand";v="8", "Chromium";v="100"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.127 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0aa3008404fa412d82219283003200ef.web-security-academy.net/login
Accept-Encoding: gzip, deflate
Accept-Language: es-ES,es;q=0.9
```

Response:

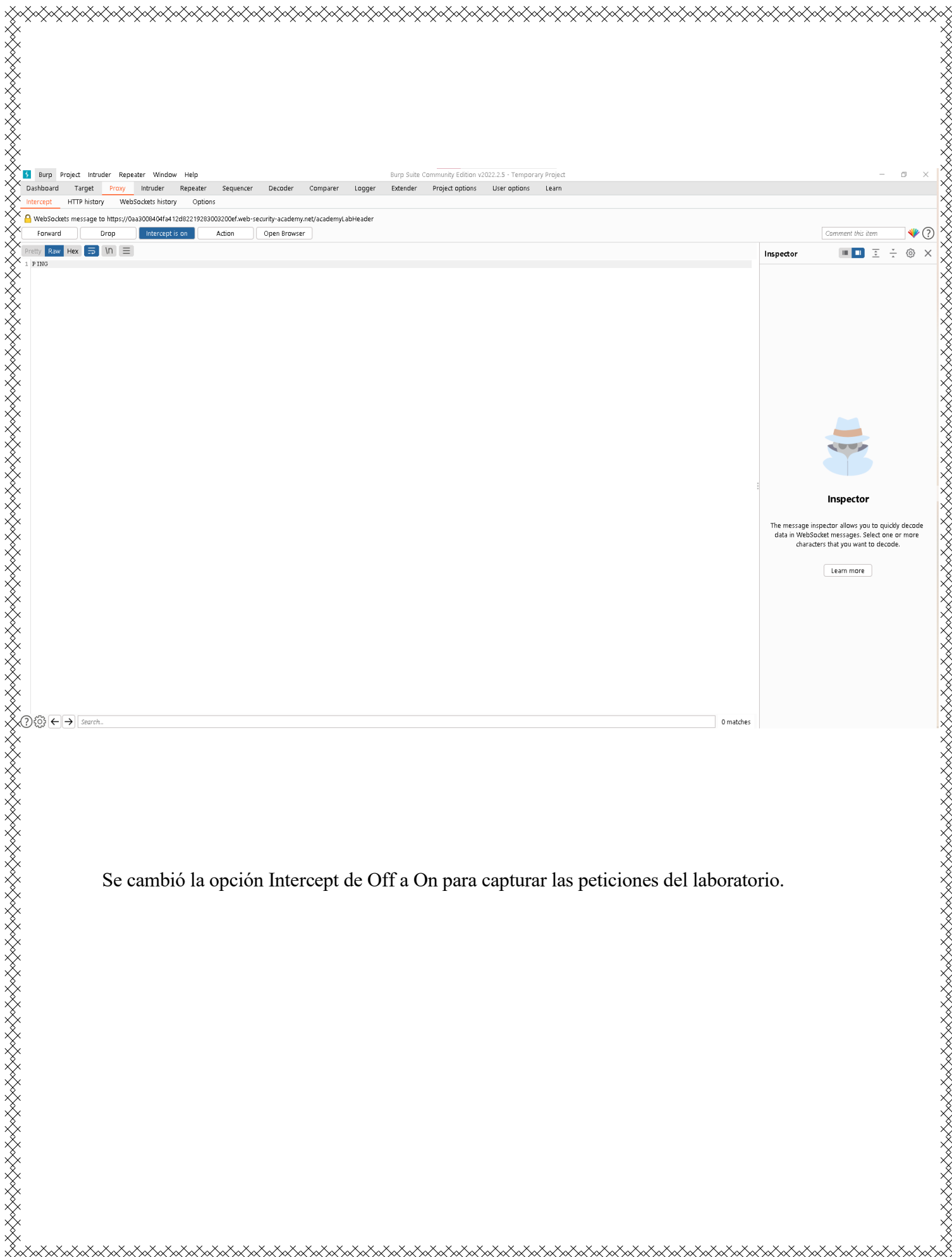
```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Cache-Control: no-cache
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 3214
6
7 <!DOCTYPE html>
8 <html>
9 <!--LAB_HEAD_START-->
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
12 <link href=/resources/css/labs.css rel=stylesheet>
13 <title>
14 Modifying serialized objects
15 </title>
16 </head>
17 <script src=/resources/labheader/js/labHeader.js>
18 </script>
19 <!--LAB_HEADER_START-->
20 <div id=academyLabHeader>
21 <section class=academyLabBanner>
22 <div class=container>
23 <div class=logo>
24 </div>
25 <div class=title-container>
26 <h2>
27 Modifying serialized objects
28 </h2>
29 <a class=link-back href=
30 https://portswigger.net/web-security/deserialization/exploiting/lab-deseria
31 lization-modifying-serialized-objects>
32 Back to top
33 </a>
34 </div>
35 <div class=widgetcontainer-lab-status is-notsolved>
36 <span>
37 LAB
38 </span>
39 <p>
40 Not solved
41 </p>
42 <span class=lab-status-icon>
43 </span>
44 </div>
```

Inspector:

- Request Attributes: 2
- Request Query Parameters: 1
- Request Body Parameters: 0
- Request Cookies: 1
- Request Headers: 19
- Response Headers: 4

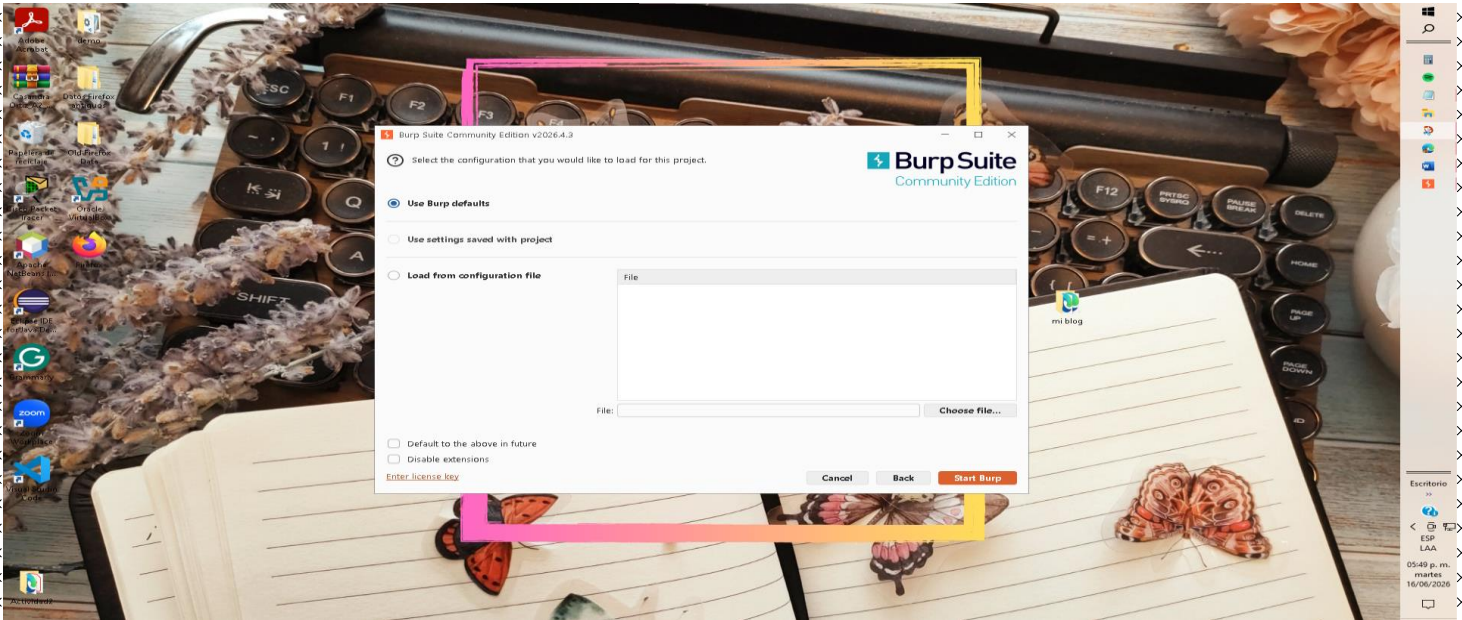
3,347 bytes | 194 millis

Se revisó la información de la solicitud y la respuesta obtenida dentro de Burp Suite.

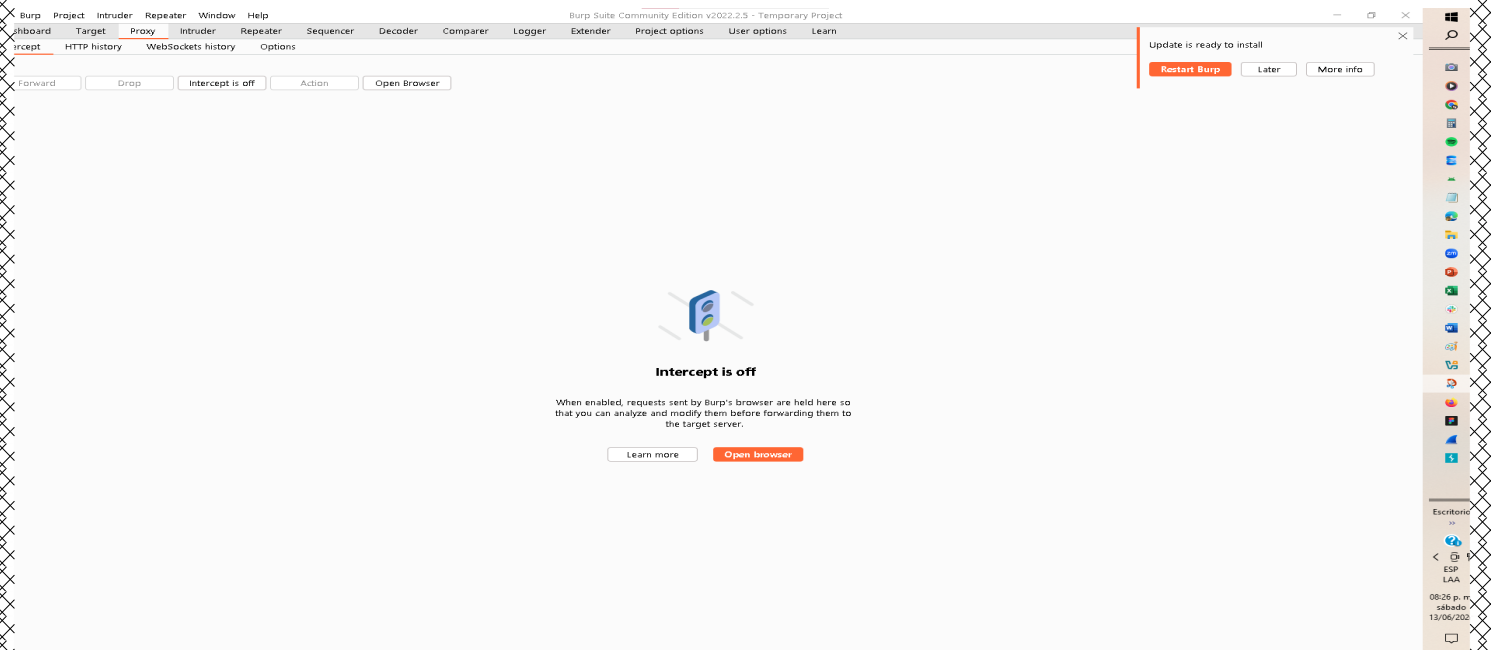


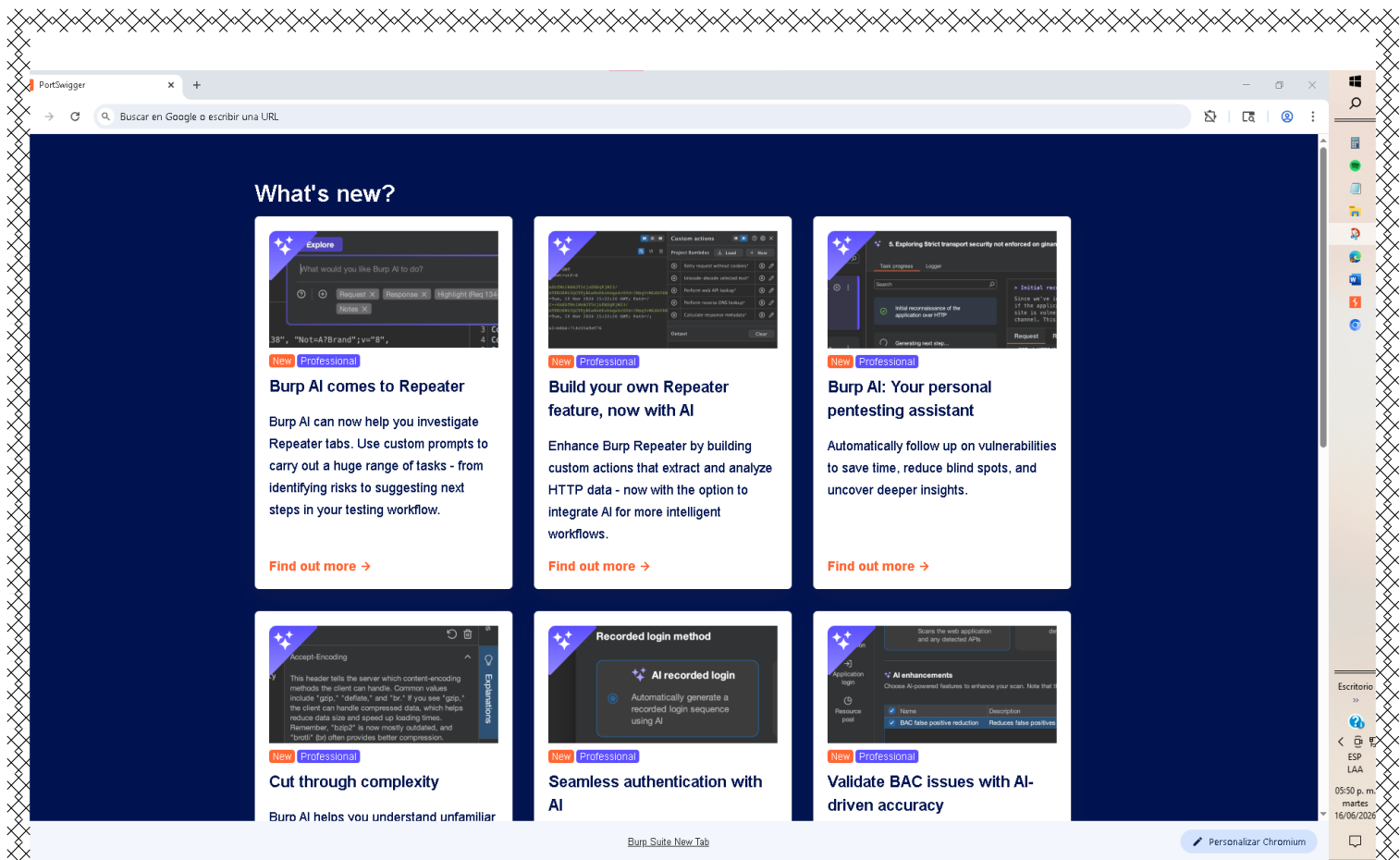
Se cambió la opción Intercept de Off a On para capturar las peticiones del laboratorio.

- Etapa 3:
 - Ataque al sitio

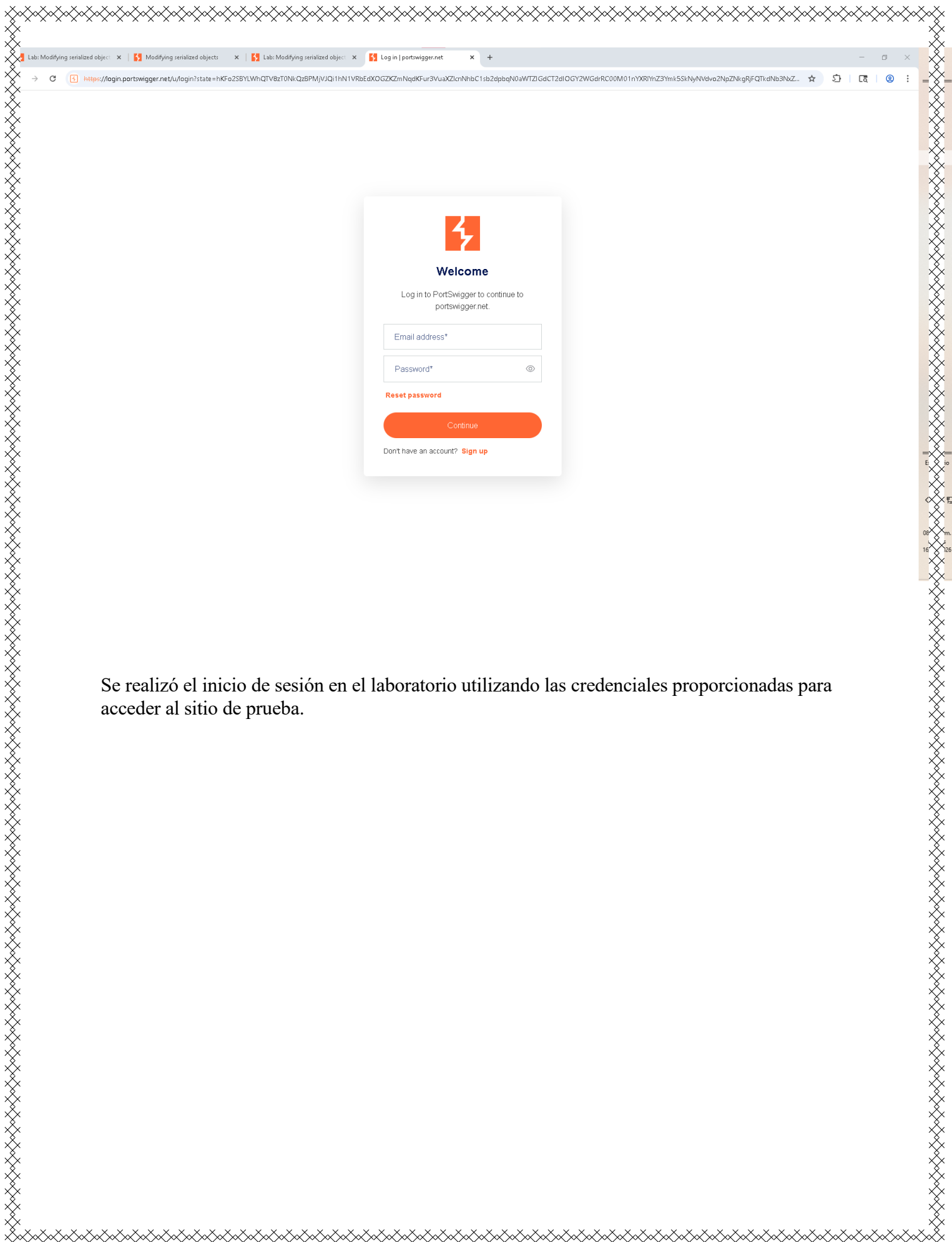


Se creó un proyecto temporal en Burp Suite Community Edition para comenzar la práctica y configurar el entorno de trabajo necesario para el análisis del laboratorio.

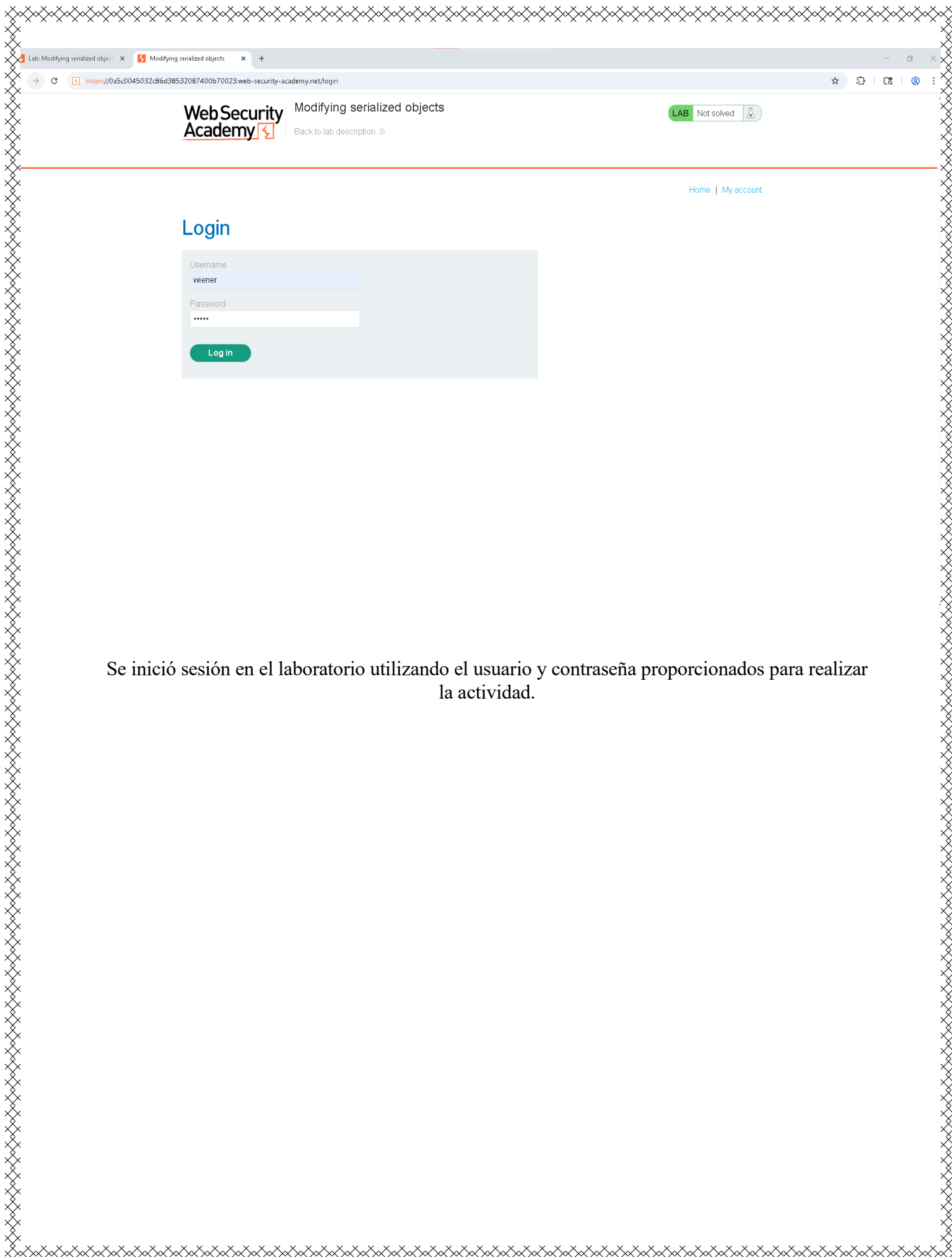




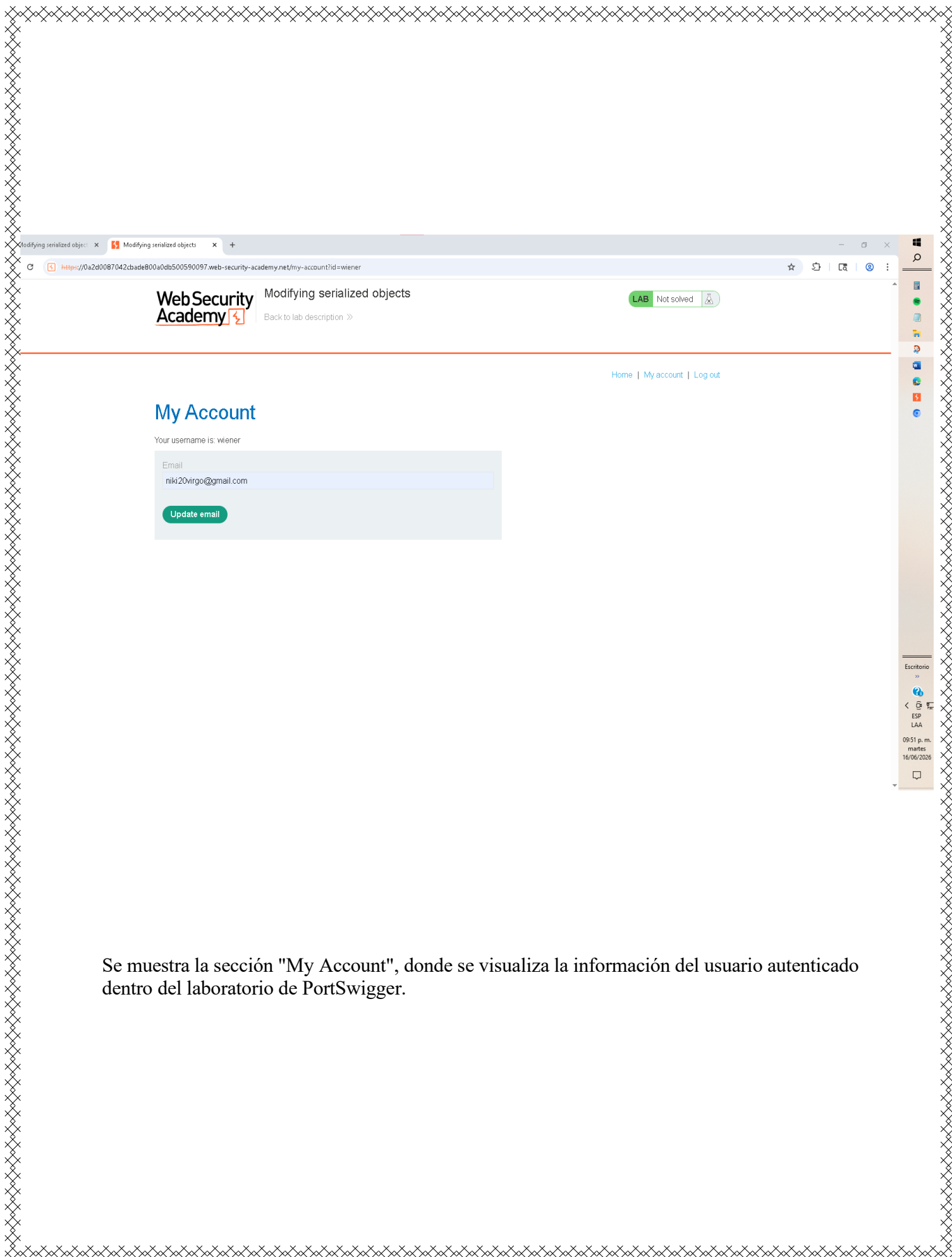
Se muestra la pantalla principal de Burp Suite con las novedades y herramientas disponibles para realizar pruebas de seguridad en aplicaciones web.



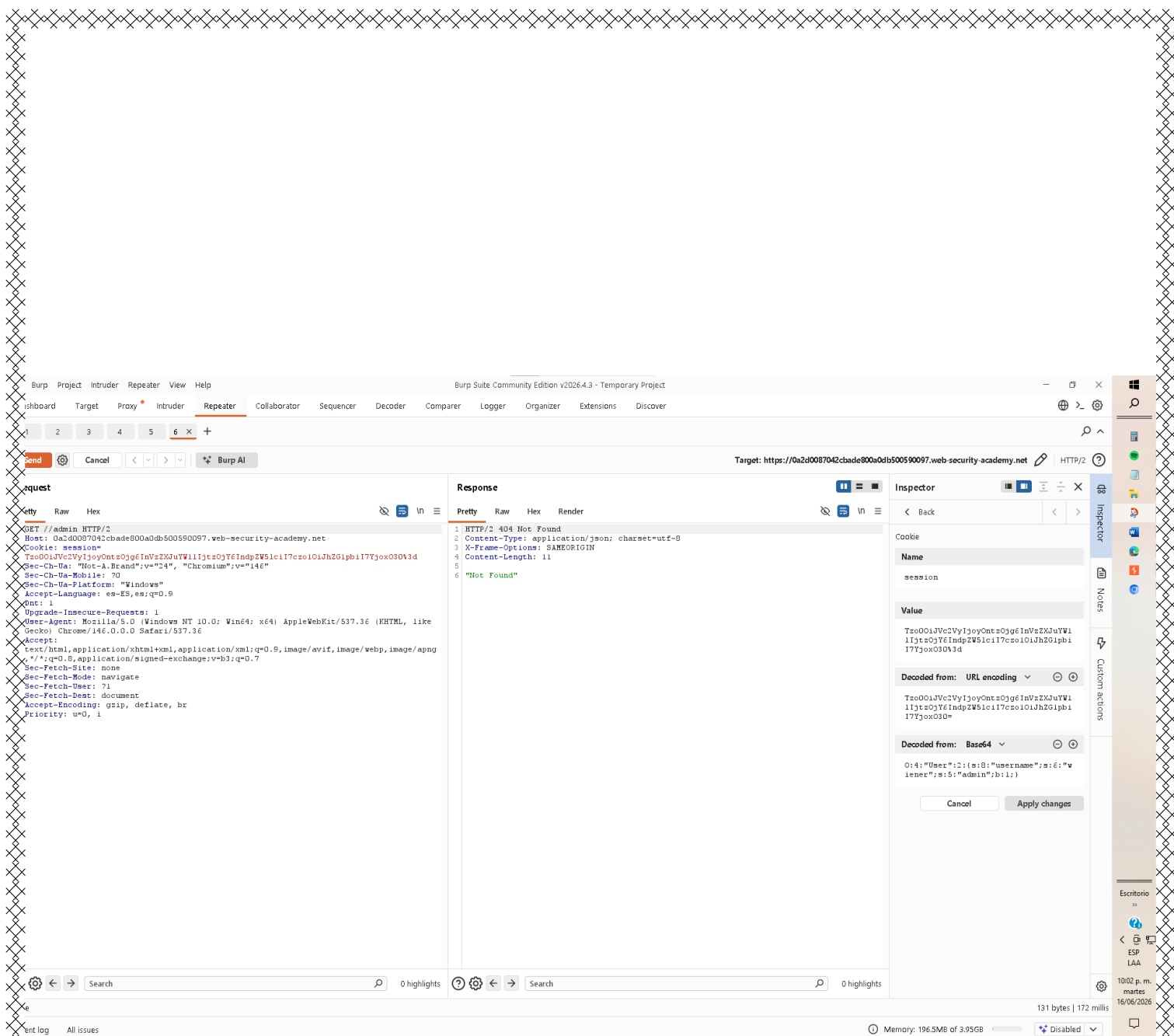
Se realizó el inicio de sesión en el laboratorio utilizando las credenciales proporcionadas para acceder al sitio de prueba.



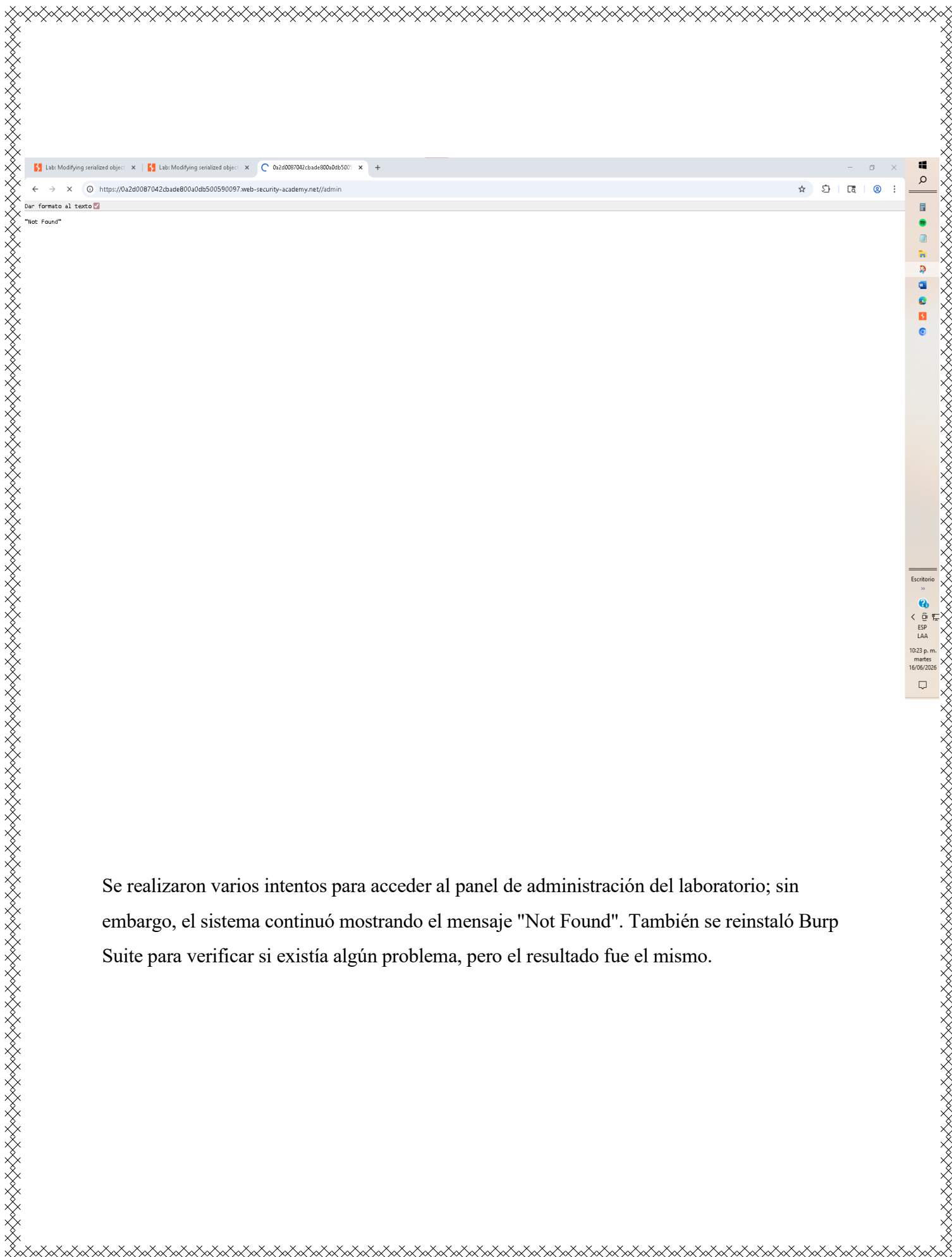
Se inició sesión en el laboratorio utilizando el usuario y contraseña proporcionados para realizar la actividad.



Se muestra la sección "My Account", donde se visualiza la información del usuario autenticado dentro del laboratorio de PortSwigger.



Se analizó una solicitud capturada en Burp Suite mediante la herramienta Repeater, observando los datos de la petición y la respuesta generada por el servidor.



Se realizaron varios intentos para acceder al panel de administración del laboratorio; sin embargo, el sistema continuó mostrando el mensaje "Not Found". También se reinstaló Burp Suite para verificar si existía algún problema, pero el resultado fue el mismo.

Conclusión

En conclusión, Cross Site Scripting (XSS) es una de las vulnerabilidades más comunes en las aplicaciones web y representa un riesgo importante para la seguridad de la información. A través de esta actividad se logró comprender cómo un atacante puede aprovechar la falta de validación de datos para ejecutar código malicioso dentro de una página web. También se identificó la importancia de proteger las credenciales, las sesiones y los datos personales de los usuarios para evitar accesos no autorizados. Durante la práctica se utilizaron herramientas como Burp Suite para observar y analizar el comportamiento de las solicitudes enviadas entre el navegador y el servidor. Esto permitió conocer mejor el funcionamiento de las aplicaciones web y la manera en que pueden presentarse diferentes vulnerabilidades. Finalmente, se concluye que aplicar medidas de seguridad, validar correctamente la información recibida y realizar auditorías constantes ayuda a reducir riesgos y fortalecer la protección de los sistemas informáticos frente a posibles ataques.

Referencia:

Qué es un ataque de XSS o Cross-Site Scripting. (2021, 28 septiembre).

<https://www.welivesecurity.com/la-es/2021/09/28/que-es-ataque-xss-cross-site-scripting/>